

Working With Permissions

How access is granted, how data is scoped to an institution, and how to give a role exactly what it needs — and nothing more.

116

PERMISSION KEYS

9

CATEGORIES

2

ACCESS LAYERS

What this covers

1. The two questions every request answers — the model everything else follows
2. Data scope and the master control — `data.master`
3. How a request is decided — resolution order, front and back
4. Granting permissions in the Roles screen — step by step
5. Assigning a role to a person
6. Common tasks — recipes for the requests you will actually get
7. Troubleshooting — "why can't they see it?"
8. Full permission reference — all 116 keys by category
9. Role baselines
10. For developers — adding a key, scoping a new endpoint

READ THIS FIRST

Granting a permission does **not** decide which records someone sees. A trainer given `students.read` sees the learners in the subjects assigned to them — not every learner in the system. That second layer is described in section 2, and it is the part most often missed.

The two questions every request answers

Access in LAD is decided in two independent steps. Keeping them separate is what lets you hand a trainer an administrative screen without also handing them the whole country's data.

LAYER 1 · PERMISSION KEY

May you open this?

A key such as `students.read` decides whether a screen, a route, and its API endpoint respond to you at all. Keys are granted per role on the Roles screen.

Fails → `403 Permission denied`

LAYER 2 · DATA SCOPE

Whose records do you see?

Once inside, the rows returned are filtered to your institution, and — if you are a trainer — to the subjects assigned to you. Enforced in the database query, not in the page.

Out of scope → the record is simply not there (`404`)

Why the second layer matters

Layer 2 is applied as a filter on the SQL query. A record outside your scope is not fetched, so it cannot be leaked by a page that forgets to hide it, by a direct API call, or by guessing a record's ID in the URL. Both the list endpoints and the single-record endpoints apply it.

Who	Sees by default
Administrator	Everything. The <code>*</code> wildcard carries master data access implicitly.
Manager / custom staff role	Every record belonging to their own institution.
Trainer	Their own institution, narrowed again to the subjects assigned to them — those subjects' modules, courses, learners, marks, and documents.
Student	Themselves. A learner granted <code>students.read</code> still sees only their own record, and cannot enumerate classmates.

THE INSTITUTION CHAIN

Scope is resolved by walking the academic structure:

`Institution` → `Department` → `Course` → `Module` → `Subject` . People are matched on their `institution_id` , and learners additionally through the course they are enrolled on — so a learner whose account carries no institution is still correctly placed by their course.

Data scope and the master control

One key lifts both confinements at once.

`data.master` — View Master Data

Found on the **Data Scope** tab of the role editor, labelled “View Master Data — *all institutions, all trainers' subjects*”. A role holding it reads across every institution and across every trainer's teaching load, on every screen it can already open.

Without <code>data.master</code>	With <code>data.master</code>
Institutions list shows only your own	Shows every institution
A trainer sees their assigned subjects	Sees all subjects
A trainer sees learners in those subjects	Sees all learners
Reports and dashboards cover your institution	Cover the whole group
Announcements reach your institution	Announcements are visible across institutions

GRANT IT DELIBERATELY

`data.master` is the only supported way to see beyond your own scope, and it is deliberately a single, visible switch rather than a side effect of any other key. Give it to group-level auditors, multi-campus registrars, and quality assurance staff. Do not give it to a role simply because a screen looks empty — an empty screen is usually a missing *assignment*, not a missing permission. See Troubleshooting.

The two alert keys

The same tab carries the alerting keys, because what they surface is scoped the same way.

Key	Effect
<code>alerts.read</code>	See open performance and attendance alerts for learners in scope. A learner reaches their own alerts through <code>notifications.read</code> without needing this.
<code>alerts.manage</code>	Re-run the evaluation and resolve alerts. <code>reports.student.write</code> also satisfies this.

Alerts open when a learner's average falls below **50%** or attendance falls below **75%** over the last 90 days, and resolve automatically on recovery. Raising one notifies the learner and the trainers who teach them.

How a request is decided

On the server

Every guarded endpoint runs the same check before it touches data:

- 1 **Is there a valid token?** No → 401 .
- 2 **Does the role hold `*`?** Yes → the key check passes.
- 3 **Is the role named exactly `Admin`?** Yes → the key check passes.
- 4 **Is the key present and set to `true`?** No → 403 .
- 5 **Scope the query.** Institution filter, then trainer-subject filter, then self-filter for learners — unless `data.master` lifts them.

ONLY TRUE COUNTS

A key must be exactly boolean `true` . A key set to `false` , or absent, or holding a string, denies. The role editor removes a key from the object entirely when you untick it, which is the same thing as denying it.

Admin-flavoured endpoints

Some administrative endpoints accept a specific key as an alternative to being an administrator. That is what allows a trainer or manager to be handed one admin screen from the Roles page without being made an admin. An endpoint guarded with no key at all stays administrator-only.

In the dashboard

The sidebar, the routes, and the page bodies all check the same keys, so a screen a role cannot use is not offered. Two behaviours are worth knowing:

- A check for `reports.student.term` is also satisfied by `reports.student.term.view` or `...read` . That is why the reference lists both the parent key and its `.view` / `.print` / `.export` variants.
- Administrator status is decided by the wildcard or the role name — never by the account type. An account without a learner or trainer profile is reported as type `admin` by the API, and a Manager falls into that bucket; treating it as a wildcard would hand non-admin staff the whole dashboard.

THE INTERFACE IS A CONVENIENCE, NOT THE BOUNDARY

Hiding a button does not protect data. Every check above is enforced again on the server, and the data scope is applied in SQL. When you review access, review the role's keys — not what the screen happens to show.

Granting permissions in the Roles screen

Open **People** → **Roles**. You need `roles.read` to see the screen, `roles.create` to add a role, and `roles.update` to change one.

- 1 **Add Role**, or **Edit** beside an existing one.
- 2 **Name the role** for the job, not the person — *Head of Department, Exams Officer, Quality Assurance*. Roles are shared by everyone who holds them.
- 3 **Leave "Full Access (Wildcard)" off** unless this really is a system administrator. Ticking it replaces every other selection with `*`, which grants all present *and future* permissions plus master data access.
- 4 **Work through the tabs**. Each tab shows a badge with the number of keys granted in it, so you can see at a glance where access has been given. *Select all <tab>* ticks the whole category; press it again to clear it.
- 5 **Start with the Admin Access tab** if the role needs an administrative screen. Each key there opens one destination — its sidebar entry, its route, and its API together.
- 6 **Decide the data scope last**. On the **Data Scope** tab, grant `data.master` only if this role genuinely works across institutions.
- 7 **Save**. Changes apply the next time the affected users sign in, because permissions are read into the session at login.

A WORKING HABIT

Grant the narrowest set that lets someone finish their job, then widen on request. A permission that was never needed is far easier to add later than one that was granted broadly and quietly relied upon.

Read, then write

Write and delete keys are not much use without their matching read key — someone who can update a course but not read one has nowhere to work from. Grant `...read` alongside `...create`, `...update`, or `...delete`.

Assigning a role to a person

- 1 Open **People** → **Users**. This needs `users.read`.
- 2 Create the user, or edit an existing one. Editing needs `users.update`.
- 3 Pick the **Role** from the dropdown — it lists every role you can read.
- 4 Set the user's **Institution**. This is what confines them under Layer 2; without it they are not placed in any institution.
- 5 Save. The person picks up the role's permissions at their next sign-in.

ONE ROLE PER PERSON

A user holds exactly one role. There is no stacking — if someone needs the union of two jobs, create a role for that combined job rather than trying to grant both.

Trainers need assignments as well as permissions

A trainer's visible data comes from their **subject assignments**, not from their permission keys. A newly created trainer with a full set of teaching permissions still sees nothing until units are assigned to them.

- 1 Open **People** → **Trainers** and choose **Assign Subjects** for the trainer. This needs `trainers.update`.
- 2 Pick a course, then a module, then one or more subjects, and assign.
- 3 The **Currently assigned units** list below shows what they hold. **Remove** takes a unit back off them — and with it, the learners, marks, and documents that unit gave them sight of.

Common tasks

Let a trainer message the learners in their subject

Grant `notifications.create` and `notifications.read`. On the Notifications screen they can then target *Students in course / module / subject* or *by enrolment year*. The school-wide and by-role targets are refused for a scoped trainer by the server, so this grant cannot become a broadcast.

Give a manager oversight of one campus

Grant the read keys for the areas they oversee plus `admin.analytics.read`, and set their institution on the Users screen. Leave `data.master` off — that is what keeps them to their own campus.

Give a group auditor read-only sight of every campus

Grant `data.master`, the read keys for what they must review, and no create, update, or delete keys at all. They will see every institution's data and be unable to change any of it.

Open one admin screen to a trainer

Grant just that screen's key from the **Admin Access** tab — for example `attendance.report.view` for the school-wide attendance report. The sidebar entry, the route, and the API all unlock together, and the data stays scoped to their institution.

Let someone upload marks but not alter existing ones

Grant `scores.create` and `scores.read`. Withhold `scores.update`, `admin.scores.update`, and `admin.scores.delete`.

Take access away

Untick the keys on the role — which affects everyone holding it — or move the individual to a narrower role on the Users screen. To remove a trainer's sight of a class without touching their permissions, remove the unit assignment instead.

Troubleshooting

Symptom	Where to look
"You do not have permission to view this"	Layer 1. The role is missing that screen's key. Check the matching tab in the role editor.
The screen opens but the list is empty	Layer 2 — this is the common case. For a trainer, check their subject assignments first. For anyone else, check the institution set on their user record.
A trainer sees no learners, marks, or documents	They have no units assigned. Trainers → Assign Subjects. Permissions are not the problem.
A change to a role had no effect	Permissions load at sign-in. Have the person sign out and back in.
A staff member sees another campus's data	Either their role holds <code>data.master</code> , or it holds <code>*</code> , or their user record has no institution set.
The sidebar entry is missing	The nav entry carries the same key as the route and the API. Grant the key and it appears.
A learner can see more than their own record	Check whether their role was given <code>data.master</code> . Nothing else lifts the self-only scope for a learner.
An announcement did not reach anyone	Announcements deliver on publish. An unpublished one sits waiting; publish it to fan it out.

A DIAGNOSTIC ORDER THAT WORKS

Ask in this order: **(1)** Does the role hold the key? **(2)** Is the person's institution set? **(3)** If they are a trainer, do they hold the unit? **(4)** Have they signed in since the change? Nearly every access question resolves at one of those four.

Full permission reference

Every key the role editor renders, grouped as it appears on screen. A key shown in more than one category is the same key — ticking it anywhere grants it everywhere.

Data Scope 3 keys

Key	What it grants
<code>data.master</code>	View Master Data — all institutions, all trainers' subjects
<code>alerts.read</code>	Read Performance & Attendance Alerts
<code>alerts.manage</code>	Run & Resolve Alerts

Admin Access 16 keys

Key	What it grants
<code>admin.analytics.read</code>	Admin Dashboard & System Analytics
<code>notifications.read</code>	Notifications Console
<code>attendance.report.view</code>	School-Wide Attendance Report
<code>roles.read</code>	Open Roles & Permissions
<code>institutions.read</code>	Open Institutions
<code>departments.read</code>	Open Departments
<code>admin.scores.read</code>	Open Score Management
<code>users.read</code>	Open User Management
<code>data.import</code>	Open Data Import
<code>documents.read</code>	Open Document Library
<code>practical.assessments.manage</code>	Author Practical Assessments
<code>online_exams.manage</code>	Author Online Exams
<code>reports.admin.compliance</code>	Open Compliance Report
<code>reports.practical.assessment</code>	Open Practical Assessment Reports
<code>reports.admin.pass_rate</code>	Open Exam Results & Analysis
<code>reports.admin.enrolment</code>	Open Enrolment Report

People 16 keys

Key	What it grants
<code>users.create</code>	Create Users
<code>users.read</code>	Read Users
<code>users.update</code>	Update Users
<code>users.delete</code>	Delete Users
<code>roles.create</code>	Create Roles
<code>roles.read</code>	Read Roles
<code>roles.update</code>	Update Roles
<code>students.create</code>	Create Students
<code>students.read</code>	Read Students
<code>students.update</code>	Update Students
<code>students.delete</code>	Delete Students
<code>trainers.create</code>	Create Trainers
<code>trainers.read</code>	Read Trainers
<code>trainers.update</code>	Update Trainers
<code>trainers.delete</code>	Delete Trainers
<code>data.import</code>	Import Data

Institution 20 keys

Key	What it grants
<code>institutions.create</code>	Create Institutions
<code>institutions.read</code>	Read Institutions
<code>institutions.update</code>	Update Institutions
<code>institutions.delete</code>	Delete Institutions
<code>departments.create</code>	Create Departments
<code>departments.read</code>	Read Departments
<code>departments.update</code>	Update Departments
<code>departments.delete</code>	Delete Departments

Key	What it grants
<code>courses.create</code>	Create Courses
<code>courses.read</code>	Read Courses
<code>courses.update</code>	Update Courses
<code>courses.delete</code>	Delete Courses
<code>modules.create</code>	Create Modules
<code>modules.read</code>	Read Modules
<code>modules.update</code>	Update Modules
<code>modules.delete</code>	Delete Modules
<code>subjects.create</code>	Create Subjects
<code>subjects.read</code>	Read Subjects
<code>subjects.update</code>	Update Subjects
<code>subjects.delete</code>	Delete Subjects

Academics 14 keys

Key	What it grants
<code>scores.create</code>	Create Scores (incl. Bulk Marks Upload)
<code>scores.read</code>	Read Scores
<code>scores.update</code>	Update Scores
<code>admin.scores.read</code>	Open Score Management
<code>admin.scores.update</code>	Edit Any Score
<code>admin.scores.delete</code>	Delete Any Score
<code>student_subjects.create</code>	Assign Student Subjects
<code>student_subjects.read</code>	Read Student Subjects
<code>student_subjects.delete</code>	Remove Student Subjects
<code>trainer_subjects.read</code>	Read Trainer Subjects
<code>analytics.read</code>	Read Analytics
<code>admin.analytics.read</code>	Read Admin Analytics
<code>practical.assessments.manage</code>	Author Practical Assessments

Key	What it grants
<code>online_exams.manage</code>	Author Online Exams

Attendance 4 keys

Key	What it grants
<code>attendance.create</code>	Create Sessions
<code>attendance.read</code>	Read Attendance
<code>attendance.write</code>	Manage Sessions (End/Regenerate)
<code>attendance.report.view</code>	Open School-Wide Attendance Report

Content 9 keys

Key	What it grants
<code>documents.create</code>	Upload Documents
<code>documents.read</code>	Read Documents
<code>documents.delete</code>	Delete Documents
<code>announcements.create</code>	Create Announcements
<code>announcements.read</code>	Read Announcements
<code>notifications.create</code>	Create Notifications
<code>notifications.read</code>	Read Notifications
<code>notifications.update</code>	Update Notifications
<code>notifications.delete</code>	Delete Notifications

Reports 44 keys

Key	What it grants
<code>reports.student.term.view</code>	View Student Report Cards
<code>reports.student.term.print</code>	Print Student Report Cards
<code>reports.student.term.export</code>	Export Student Report Cards
<code>reports.student.transcript.view</code>	View Student Transcripts
<code>reports.student.transcript.print</code>	Print Student Transcripts

Key	What it grants
<code>reports.student.transcript.export</code>	Export Student Transcripts
<code>reports.student.attendance.view</code>	View Student Attendance Reports
<code>reports.student.attendance.print</code>	Print Student Attendance Reports
<code>reports.student.attendance.export</code>	Export Student Attendance Reports
<code>reports.student.discipline.view</code>	View Student Discipline Reports
<code>reports.student.discipline.print</code>	Print Student Discipline Reports
<code>reports.student.discipline.export</code>	Export Student Discipline Reports
<code>reports.student.write</code>	Write Reports for Students
<code>reports.class.performance.view</code>	View Class Performance Reports
<code>reports.class.performance.print</code>	Print Class Performance Reports
<code>reports.class.performance.export</code>	Export Class Performance Reports
<code>reports.class.at_risk.view</code>	View Class At-Risk Reports
<code>reports.class.at_risk.print</code>	Print Class At-Risk Reports
<code>reports.class.at_risk.export</code>	Export Class At-Risk Reports
<code>reports.teacher.syllabus.view</code>	View Syllabus Coverage Reports
<code>reports.teacher.syllabus.print</code>	Print Syllabus Coverage Reports
<code>reports.teacher.syllabus.export</code>	Export Syllabus Coverage Reports
<code>reports.teacher.attendance.view</code>	View Teacher Attendance Reports
<code>reports.teacher.attendance.print</code>	Print Teacher Attendance Reports
<code>reports.teacher.attendance.export</code>	Export Teacher Attendance Reports
<code>reports.teacher.appraisal.view</code>	View Teacher Appraisal Reports
<code>reports.teacher.appraisal.print</code>	Print Teacher Appraisal Reports
<code>reports.teacher.appraisal.export</code>	Export Teacher Appraisal Reports
<code>reports.admin.pass_rate.view</code>	View Admin Exam Results
<code>reports.admin.pass_rate.print</code>	Print Admin Exam Results
<code>reports.admin.pass_rate.export</code>	Export Admin Exam Results
<code>reports.admin.enrolment.view</code>	View Admin Enrolment Reports
<code>reports.admin.enrolment.print</code>	Print Admin Enrolment Reports

Key	What it grants
<code>reports.admin.enrolment.export</code>	Export Admin Enrolment Reports
<code>reports.admin.safeguarding.view</code>	View Safeguarding Reports
<code>reports.admin.safeguarding.print</code>	Print Safeguarding Reports
<code>reports.admin.safeguarding.export</code>	Export Safeguarding Reports
<code>reports.admin.compliance.view</code>	View Compliance Reports
<code>reports.admin.compliance.print</code>	Print Compliance Reports
<code>reports.admin.compliance.export</code>	Export Compliance Reports
<code>reports.practical.assessment</code>	Open Practical Assessment Reports
<code>reports.practical.assessment.view</code>	View Practical Assessment Reports
<code>reports.practical.assessment.print</code>	Print Practical Assessment Reports
<code>reports.practical.assessment.export</code>	Export Practical Assessment Reports

Feedback 3 keys

Key	What it grants
<code>feedback.trainer.submit</code>	Submit Feedback About Trainers
<code>feedback.trainer.view</code>	View All Trainer Feedback
<code>feedback.trainer.moderate</code>	Reveal Identity On Anonymous Feedback

Role baselines

The sets below are the intended starting point for each standard role, as defined in the dashboard source. Use them as a checklist when building a role by hand — the editor does not apply them for you.

Admin System · wildcard

Holds the wildcard `*` — every permission, present and future, plus master data access. Reserve it for genuine system administrators.

Student Academic · 12 keys

students.read students_view_own_subjects student_subjects.read scores.read announcements.read
notifications.read documents.read attendance.read analytics.read reports.student.term.view
reports.student.attendance.view feedback.trainer.submit

Trainer Academic · 38 keys

trainers.read trainers.update students.read subjects.read courses.read scores.read
scores.create scores.update modules.read announcements.read announcements.create
notifications.read notifications.create documents.read documents.create documents.delete
attendance.create attendance.read attendance.write analytics.read trainer_subjects.read
alerts.read alerts.manage reports.class.performance.view reports.class.performance.print
reports.class.performance.export reports.teacher.syllabus.view reports.teacher.attendance.view
reports.student.discipline.view reports.student.discipline.print reports.student.discipline.export
reports.student.write practical.assessments.manage online_exams.manage
reports.practical.assessment reports.practical.assessment.view reports.practical.assessment.print
reports.practical.assessment.export

Manager Management · 29 keys

departments.read departments.update courses.read courses.update students.read trainers.read
subjects.read modules.read scores.read analytics.read announcements.read
announcements.create notifications.read notifications.create documents.read attendance.read
reports.admin.pass_rate.view reports.admin.enrolment.view reports.class.performance.view
reports.class.at_risk.view reports.student.write feedback.trainer.view alerts.read
alerts.manage admin.analytics.read attendance.report.view reports.practical.assessment
reports.practical.assessment.view reports.practical.assessment.export

For developers

Where the pieces live

File	Responsibility
<code>backend/app/routes/permissions.py</code>	Key checks and the route guards
<code>backend/app/services/scoping.py</code>	Data scope — institution, trainer, and <code>data.master</code>
<code>dashboard/src/auth/ProtectedRoute.tsx</code>	Route guards and the shared <code>hasPermission</code> helper
<code>dashboard/src/pages/RolesPage.tsx</code>	The permission catalogue rendered by the editor
<code>dashboard/src/components/layout/Sidebar.tsx</code>	Nav entries and the key each carries

Adding a permission key

- 1 Guard the endpoint with the new key.
- 2 Add the key and a plain-language label to the right tab in `PERMISSION_TABS`.
- 3 Guard the route, and the sidebar entry, with the same key so all three unlock together.
- 4 Add it to the role baselines that should hold it.
- 5 Rebuild this document — the reference is parsed from the catalogue, so the new key appears without being retyped.

Scoping a new endpoint

Apply the scope as a query filter, never as a post-filter on results. The helpers return a narrowed query, so a route that calls one cannot leak; a route that forgets will.

```
from ..services.scoping import scope_students, can_access_student

# Lists – scope the query before it runs
students = scope_students(db.session.query(Student), user).all()

# Single records – the same scope, or the caller gets a 404
if not student or not can_access_student(user, student.id):
    return {"error": "Student not found"}, 404
```

Return `404` rather than `403` for an out-of-scope record: a `403` confirms the record exists, which is itself a disclosure.

ONE INCONSISTENCY TO KNOW ABOUT

The key check treats the role name `Admin` as a wildcard, matched case-sensitively. The separate administrator test accepts `admin` or `super admin` in any case. A role named *Super Admin* therefore passes the administrator test but must rely on holding `*` to satisfy key checks — which the seeded super-admin role does. Prefer granting `*` explicitly over depending on a role's name.

Marks arithmetic

Unrelated to permissions but frequently touched alongside them: a score becomes a percentage as $(x / y) * 100$ when the assessment records a total, and $(x / 100) * 100$ when it does not. Average over percentages, never over raw marks, so a paper out of 40 and one out of 100 stay comparable.

Rebuilding this document

```
python3 docs/build_permissions_guide.py
```